

stalin

The First Five-Year Plan as a container

Documentation

September 1, 2026

Abstract

stalin is a resource-allocation game about Soviet industrialisation, built on the container algebra of Neil Ghani’s *Containers for Typed Agentic AI*. It runs as five HTTP servers exchanging JSON. This document describes the game and how to win it; revises the parts of the framework it uses; draws the game as a composite of the four combinators; and then sets out what the implementation actually guarantees — which turns out to be less than it looks, in a way that is the game’s own subject.

Contents

1	The game	3
1.1	The country you are given	3
1.2	What you are trying to do	3
1.3	A year is two turns	3
1.4	The score	3
2	Strategies	3
2.1	Reading the table	4
2.2	The plan the game is built around, and the one that beats it	4
3	Ghani’s containers, revised	4
3.1	§3 — a container is a dependent type	5
3.2	§4 — morphisms, or how one container calls another	5
3.3	§5 — the four combinators	5
4	The game, as a composite	6
4.1	Where the sum is, and is not	6
4.2	Two that were removed	6
4.3	One that broke, and how	6
5	The implementation	7
5.1	Five servers, seven containers	7
5.2	What a hop actually is	7
5.3	JSON, and what it discards	7
6	What the compiler enforces	8
6.1	Faking containers: the graph presentation	8
6.2	Two traps in that encoding	8
6.3	What this buys	8
7	What must be checked at run time	9
7.1	The tag is a claim, not a fact	9
7.2	Validate, do not cast	9
7.3	And the replies, coming back	10

8	What is not guaranteed at all	10
9	Summary of the guarantees	11

1 The game

You are not Stalin. You are the Plan — Gosplan, in a building in Moscow, deciding each year what a hundred and twenty million people will do with themselves. You issue five plans, and then you are judged twice.

1.1 The country you are given

It grows grain and almost nothing else. In 1928 there are 120 people: 105 on the land, 8 in a small steel mill, 7 on a railway that reaches some of the places it should. There are no tractors, no foreign engineers, 10 tonnes of steel and no hard currency. Everyone eats, whether or not they grow anything.

1.2 What you are trying to do

Two things at once, and they pull against each other.

Mechanise. A peasant with a tractor out-produces a peasant with a back, so tractors let you take hands off the land and put them in the mills without the country going hungry. To build tractors you need a works; to build the works you need steel and a German engineer; the engineer wants hard currency; hard currency comes from selling grain abroad — grain the villages were going to eat. That circle is the game.

Survive 1941. You do not fight that war, but you pay for it here. Steel put by for armaments never becomes a tractor and never earns a rouble. It buys exactly one thing: soldiers who come home. You never conscript during the plan; in 1941 the country calls up precisely as many men as the armaments require, and no more — so the size of the army, and therefore the dead, is decided years earlier by how much steel you set aside.

1.3 A year is two turns

spring	where the hands go, and how hard the villages are squeezed
autumn	the harvest is in and graded; you dispose of it

The split is the point. In spring you fix the labour allocation and the procurement quota *before you know the weather*. A quota set in March against a harvest that fails in August is not a game mechanic; it is how a famine is administered. By autumn the grade is known — **failure, poor, adequate, bumper** — and it decides which export decisions are even available to you. Five years, ten turns.

1.4 The score

One number: how many of your people die. Starved and fallen, added together. Win the war, and lose as few as you can.

The game reports both halves separately and refuses to combine them for you beyond the sum. It also reports whether you mechanised — whether the final harvest beat 1928's with fewer hands — and does not fold that into the score at all.

2 Strategies

Seven representative plans, same seed. Letters are what a player types; a sequence of ten letters is a whole game.

strategy	letters	tractors	armaments	starved	fallen	outcome
gentle arms	AJBEBJAJAJ	2	89	5	4	won, 9 dead
brutalist mech.	CMKEBIBJNJ	22	98	15	3	won, 18 dead
no railway	KABEBIBJAJ	22	82	25	4	won, 29 dead
partial arms	CABEBINJAA	42	34	21	14	won, 35 dead
totally brutal	KJBBAJBBAJ	2	78	44	5	won, 49 dead
bought tractors	AMAMAFAMAF	18	0	3	57	lost
never armed	CAKEBINIAI	62	0	15	52	lost

2.1 Reading the table

Never arming loses. A beautiful mechanised country — 62 tractors, the largest harvest on the board — falls in 1941, because 120 men with nothing to fire cannot reach the enemy’s strength.

Buying tractors abroad is the humane road, and it is fatal. Three people starve, the fewest of any line. But imported tractors build no mill, no engineers, and no steel, so there is nothing to arm with. The cheap road is the trap.

Partial armaments wins and bleeds. 34 tonnes put by is enough to win and not enough to win cheaply: 14 fallen against the best line’s 3. The war curve is smooth, so half an army is a victory paid for in men.

The railway comes before the steel. Grain the railway cannot move rots at the sidings. A total quota with no track to carry it starves 25 people for nothing — worse than the same brutality with a railway.

The two orderings disagree. Ranked by *soldiers* lost, brutalist mechanisation is the best plan on the board: 3 fallen, because it ends holding 98 tonnes of armaments, more than any other line, and an army small enough to carry them. Ranked by *everyone* lost it comes second, because the fifteen it starved outnumber the one soldier it saved. The game holds both numbers and declines to say which is the real one.

2.2 The plan the game is built around, and the one that beats it

The design intent is that *brutalist mechanisation* should be optimal: starve the villages early, spend it on rail, engineers and tractors, then turn the surplus into an army. It is morally uncomfortable and it should win.

It does not. AJBEBJAJAJ — found by machine search over the menu, not by hand — wins for 9 dead by doing almost nothing: put hands in the mill, buy the Germans once, then smelt straight into armaments for three years. No railway, no tractors on purpose, no famine beyond the baseline five who die whatever you do.

The reason is a dependency length rather than a number:

	mill workforce, 1928–32	armaments
brutalist mechanisation	8, 8, 16, 23, 45	98, first built in 1932
gentle arms	8, 15, 22, 21, 20	89, first built in 1929

The mechanised economy matures in 1932, and then the plan ends. Its chain is rail → export → engineers → works → tractors → freed hands → mill → steel → armaments: nine links, each costing at least one turn, against ten turns in the plan. The gentle line skips six of them and spends four years arming instead of one.

This is recorded as a failing test in `calibrate.ts` rather than quietly redefined. Seven of the eight design criteria hold across every seed tried; this is the eighth.

3 Ghani’s containers, revised

Three ideas, in the order the note develops them.

3.1 §3 — a container is a dependent type

A container is a pair (P, R) :

$$\begin{array}{ll} P : \text{Type} & \text{the } \textit{prompts}, \text{ or shapes} \\ R : P \rightarrow \text{Type} & \text{the } \textit{positions}: \text{ for each prompt, the type of valid replies} \end{array}$$

and an inhabitant of the interface is a dependent function $(p : P) \rightarrow R p$. The content of that dependency is Ghani’s observation that event-driven programming “has a feature that turns out to be exactly a dependent type”: *which responses are appropriate depends on which event arrived*. Note what is absent — no loops, no callbacks, no concurrency. Strip the event loop away and the container is what remains.

A game is a container. P is the situations you can be in; $R s$ is the moves legal at s . A player is $(s : P) \rightarrow R s$. That is not an analogy: it is the same type.

3.2 §4 — morphisms, or how one container calls another

A morphism $(P, R) \rightarrow (Q, T)$ is a pair:

$$\begin{array}{ll} u : P \rightarrow Q & \textit{delegate}: \text{ turn my prompt into one for the level below} \\ f : (p : P) \rightarrow T (u p) \rightarrow R p & \textit{amalgamate}: \text{ turn their reply into mine} \end{array}$$

Control flows *down* through u ; data flows *up* through f . This is worth dwelling on, because it is exactly the shape of a command economy — and, in a game about one, that is not a coincidence but the reason the fit is good. Gosplan does not do the harvesting. It converts “feed the towns” into “Narkomzem: reap”, and converts the reply back into a number for its own ledger.

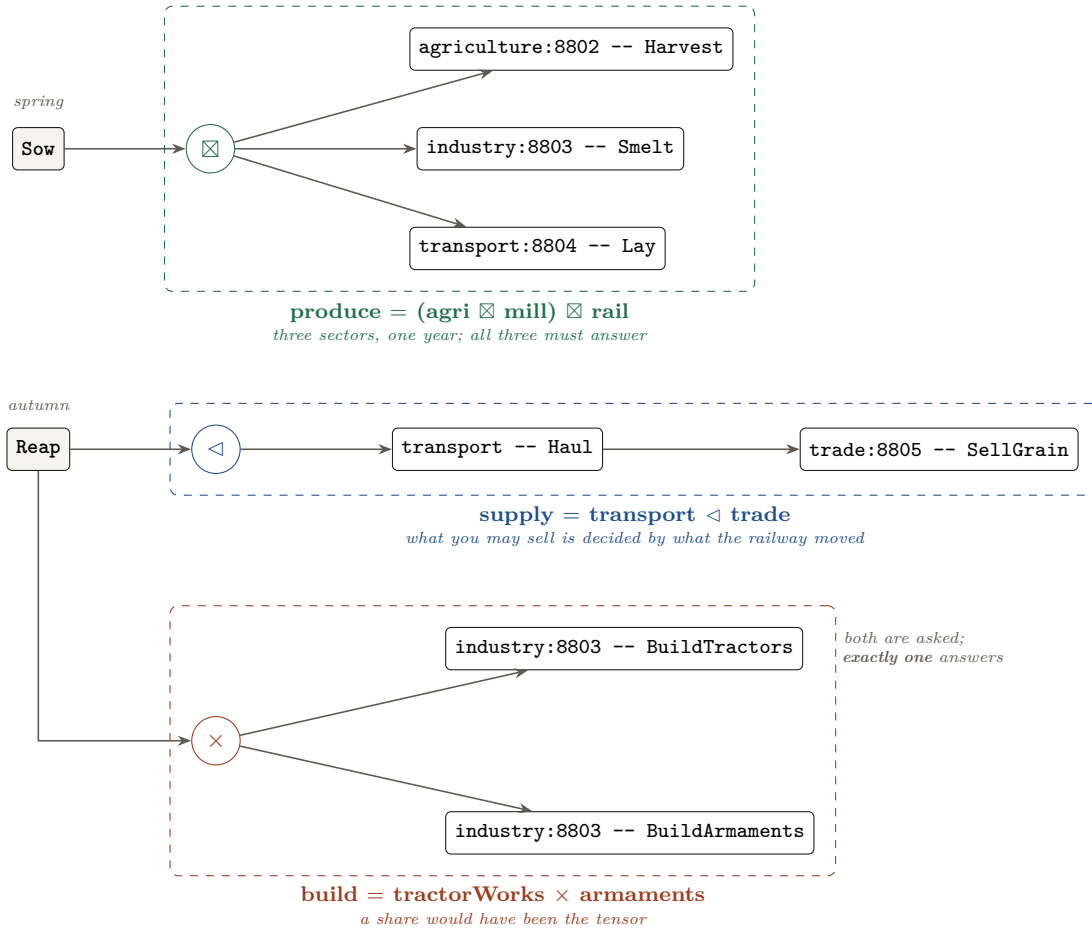
3.3 §5 — the four combinators

	shapes	positions
$+$ sum	disjoint union: one side or the other	each keeps its own
$\boxtimes$ tensor	multiply: a prompt for each side	multiply: a reply from each
$\triangleleft$ sequential	a dependent pair $(s ** R s \rightarrow S')$	a pair of replies
$\times$ product	multiply: prompt both	<i>sum</i> : exactly one answers

The distinction that does the most work here is $\boxtimes$ against $\times$. Both ask everybody. The tensor requires *all* of them to answer; the product requires exactly *one*. Same shapes, different positions.

$\triangleleft$ is the subtle one. Its shape is not a pair of prompts but a first prompt *together with a function* choosing the second from the first’s reply. So the second question is not knowable when the first is asked — which is what makes it sequential rather than parallel, and what makes it hard to type.

4 The game, as a composite



4.1 Where the sum is, and is not

$+$ is never built as a composite. There is no `sumC(...)` call in the game, and it would be an overstatement to claim one. Routing appears instead as each container’s own *shape type* — a tagged union $\{\text{Harvest}\} + \{\text{Winter}\} + \dots$ is a coproduct — and its eliminator is the `switch` in every server’s dispatch. Real work, in the place §3 always put it; but not a use of the combinator.

4.2 Two that were removed

While the railway to the port was narrow, the sale was a *product* (grain *or* steel, one contract) and became a *tensor* once enough track was laid (ship both) — rail capacity choosing between $\times$ and $\boxtimes$, §5’s own distinction used as a game progression. Selling steel was later removed from the game, on the grounds that a country exporting its steel is not industrialising. Both combinators would then have been running on a quantity that is permanently zero, so they were deleted rather than left for a diagram to claim.

4.3 One that broke, and how

Splitting the year put `produce` in spring and the haul in autumn, with a player decision between them, so the original `seqC(produce, transport)` could no longer be run in a single call. It stayed declared, stayed exported in the flow label, and silently never fired. Nothing failed to compile: `seqC(a, b)` is a perfectly good *value* whether or not anybody ever applies it. The type system verifies that a composite is well-formed, never that it is reached.

The dependent pair had not disappeared — it had moved one link down the chain, to `seqC(transport, trade)`: what you may sell depends on what the railway actually moved.

5 The implementation

5.1 Five servers, seven containers

port	process	containers it serves	holds
8801	gosplan	the Plan	the true ledger
8802	agriculture	AgricultureC	weather stream, books
8803	industry	SmelterC, TractorWorksC, ArmamentsC	books
8804	transport	TransportC	books
8805	trade	ForeignTradeC	prices, books

Five processes, seven containers. The count differs because `industry:8803` serves three. The tractor works and the armaments shop are separate containers deliberately: it is the only way the Plan can take their *product* — same steel, one contract, offer both and answer one. Had they been one container with a `build` field, that decision would have been an `if` inside a handler rather than a combinator.

The Plan is itself a container on 8801, which is why the CLI and the interactive game are just clients: they send `Sow`, `Reap`, `Status`, `Reckoning`, `Reset` and read replies. Nothing in the game logic knows whether a human or a search program is playing.

5.2 What a hop actually is

Firing a low-level prompt is not a function call. It spawns a process:

```
delegate.ts <port> <json-prompt> <depth>      # a CLI tool; stdout is the reply
```

That file *is* the delegate leg u of a morphism, made executable. It knows nothing about any container — it takes a port, a JSON prompt and a depth, and carries the text across. That ignorance is the point: u is a map of *shapes*, and shapes are exactly what survives the wire.

One consequence worth stating plainly, because it cost a debugging session. Every file type-checked and the first playthrough died immediately with `delegate to :8804 exited 1`, because `wire.ts` locates that tool with

```
new URL("./delegate.ts", import.meta.url) // resolved at run time, never checked
```

and the file had not been copied into the vendored library. The one file that makes each hop a real process was the one file the compiler could not tell us was missing. **Types hold within a process; the moment a boundary is a filesystem path or a socket, only running it tells you the truth.**

A process per hop costs about a fifth of a second, and a parameter search playing hundreds of games pays that forty times per game. `STALIN_INPROC=1` keeps the HTTP hop — same servers, same handlers, same books, same weather — and skips only the spawn: $21\times$ faster, byte-identical output. It is off by default and no game a player sees uses it.

5.3 JSON, and what it discards

JSON has six kinds of value: string, number, boolean, null, array, object. Nothing else. Two functions move between text and live values — `JSON.stringify` and `JSON.parse`. What JSON has no notion of is *types*. There is no way to write down that

```
{"tag": "Harvest", "workforce": {...}, "tractors": 20, "year": 1930}
```

is a `Harvest` prompt rather than any other object with those keys. The notation carries the data and discards the classification. Every guarantee in the next two sections is a guarantee about one side of that discard or the other.

6 What the compiler enforces

6.1 Faking containers: the graph presentation

TypeScript has no Σ -types, so a container cannot be written as a pair $(\text{Shape}, \text{Pos})$ with Pos a real dependent function. The move that makes it work is to present a container not as that pair but as its **graph** — the union of its fibres. Over a *finite* index this is exact:

$$\Sigma(s : S). B s = \bigcup_{s \in S} \{s\} \times B s$$

and every container here has a finite shape type, because a shape type is a tagged union. Conditional types distribute over unions, so the dependent lookup becomes a conditional type:

```
interface Fib<S, P> { readonly shape: S; readonly pos: P }
type Cont = Fib<unknown, unknown>;

type Shape<C extends Cont> = C extends Fib<infer S, unknown> ? S : never;

type PosAt<C extends Cont, S> =
  C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) : never;
```

A container is then a union of Fibs, one per prompt:

```
export type AgricultureC =
  | Fib<{tag:"Harvest"; workforce: Workforce; tractors: number; year: number}, HarvestReport>
  | Fib<{tag:"Winter"; ruralRations: Grain; industrialRations: Grain; year: number},
    WinterReport>
  | Fib<{tag:"Report"; year: number}, Dispatch>
  | Fib<{tag:"Census"}, CensusReturn>
  | Fib<{tag:"Reset"; seed: number}, ResetAck>;
```

`Fib` is never constructed. It is a type-level record — the object language of $\text{Fam}(\text{Set}^{\text{op}})$ — and exists only to be taken apart by `infer`.

6.2 Two traps in that encoding

The direction of the fibre test. The obvious spelling of `PosAt` is `Extract<C, Fib<S, unknown>`, which keeps the fibres assignable *to* the query. But a shape you actually hold is usually *narrower* than its fibre’s declared shape — a literal tag, a specific type. What is wanted is the fibre this shape lies in: `S extends S1`, not the reverse. Written the wrong way round, every position silently becomes `never` and nothing complains at the site of the mistake.

Conditional types are not inference sites. If `C` appears only inside `Shape<C>` and `PosAt<C, S>`, TypeScript cannot infer it: `seqC(a, b)` quietly infers `A = Cont` and every downstream position becomes `unknown`, again with no error where the error is. The fix is a phantom field:

```
export interface Runner<C extends Cont> {
  readonly fibres?: C; // never assigned; exists so C is inferable
  readonly label: string;
  run<S extends Shape<C>>(shape: S, depth: number): Promise<PosAt<C, S>>;
}
```

6.3 What this buys

The signature of `run` is the guarantee. Given a shape, the type of the reply is computed from it. Prompt an `agri` runner with a `Harvest` and you get a `HarvestReport`; with a `Census` and you get a `CensusReturn`; and the two are not interchangeable. No cast is needed at any call site, and `produced.left.left` is a `HarvestReport` because that is the fibre we prompted — not because anybody asserted it.

The same trick indexes the game's own rules. Which export decisions exist is a function of the harvest grade:

```
export type ExportChoice<G extends HarvestGrade> =
  G extends "failure" ? "none"
  : G extends "poor" ? "none" | "surplus"
  : G extends "adequate" ? "none" | "surplus" | "full"
  : "none" | "surplus" | "full" | "maximum";

exportPlan("bumper", "maximum"); // fine
exportPlan("failure", "maximum"); // does not compile
```

That is not a validation rule with a nice error message. Shipping the maximum out of a failed harvest is a *type error*: the position does not exist in that fibre. `type-tests.ts` asserts these negatively — every `@ts-expect-error` claims the next line *is* an error, and TypeScript reports an unused directive as an error in its own right, so the file fails both if a legal move is rejected and if an illegal one is accepted. A negative suite that cannot fail is worth nothing, so it was mutation-tested: widening the `"failure"` fibre by one option makes it stop compiling.

7 What must be checked at run time

7.1 The tag is a claim, not a fact

When `{ "tag": "Harvest", ... }` arrives over a socket, the server has not received a `Harvest`. It has received a stranger's assertion that this is one. `JSON.parse(body)` as `Prompt` is a cast made on behalf of a program we do not control.

A client whose tag does not match its fields is otherwise answered anyway, and — this is the part worth internalising — *there is usually no crash*. In JavaScript a missing property reads as `undefined` rather than raising, so a handler reading a field that was never sent returns normally, with HTTP 200, carrying a well-formed reply of exactly the type the container demands and a value that means nothing. A crash would have been the better outcome: loud, immediate, and it would name the line. Whether you get that or this depends on what the handler happens to do with the field. Nothing in the interface decides which.

7.2 Validate, do not cast

Every server therefore parses rather than asserts. The entry point takes `unknown` and returns the prompt type *or null*:

```
serveContainer({
  name: "agri", port: 8802,
  parse: parseAg, // (u: unknown) => AgPrompt | null
  answer: (p, depth) => ...,
});
```

and inside `serveContainer`:

```
const prompt = opts.parse(body);
if (prompt === null) {
  // The validator refused. No cast was made, so nothing ill-typed is
  // downstream of this line -- that is the whole point of validating.
  return json({ error: "not a valid prompt for this container", got: body }, 400);
}
```

The validators go through the fields the tag claims, on the parsed value rather than the text, so `year` can be required to be a number and not the string `"banana"`. Across the five servers there are 56 such field checks.

7.3 And the replies, coming back

The same problem runs the other way: a reply is also just JSON. So each leaf carries per-shape *decoders*, keyed by tag, and each entry is checked against its own concrete position type:

```
type Decoders<C extends Cont> = {
  [K in TagOf<Shape<C>>]: (u: unknown) => PosAt<C, Extract<Shape<C>>, {tag: K}>> | null;
};

const agri = leaf<AgricultureC>("agri", AGRI, {
  Harvest: (u) => rec(["grain", "grade", "perWorker", "withoutTractors"], u),
  Winter: (u) => rec(["fed", "dead", "shortfall"], u),
  Report: (u) => rec(["year", "quotaMet", "output", "note"], u),
  Census: (u) => rec(["workforce", "stocks", "trueOutput"], u),
  Reset: (u) => rec(["ok"], u),
});
```

`Decoders<C>` is a mapped type over the tags, so **the compiler will not let you forget a fibre**. Deleting the `SellSteel` decoder while its fibre still existed in `ForeignTradeC` was a compile error — the fibre and its handler are one fact stated twice, and the type system insists they agree. That is what made removing a capability from the game safe rather than archaeological.

`rec` is deliberately shallow: it confirms the keys are present and then casts. It does not check that `grain` is a number, or that `grade` is one of four strings. The single cast in `leaf` is:

```
const value = decode(raw);
if (value === null) throw new Error(`${label}/${tag}: reply did not validate`);
return value as PosAt<C, S>;
```

This is the seam. Everything above it is checked by the compiler; everything below it is bytes. The cast is confined to one line of one library function, which is the most that can be asked — but it is a cast, and calling it anything else would be dishonest.

8 What is not guaranteed at all

Every commissariat holds two things the Plan cannot see: a hidden efficiency, and how much it inflates its dispatches.

```
record(year: number, quota: number, delivered: number): Dispatch {
  const gap = Math.max(0, quota - delivered);
  const reported = delivered + gap * this.bias * (0.7 + 0.6 * this.noise());
  ...
}
```

A commissariat that met its quota reports honestly; one that missed closes the gap by the fraction it dares. Gosplan keeps the true ledger, because the grain physically exists — what diverges is the *report*, and the player plans against the report.

This is §6's caveat about the LLM oracle, relocated into the subject matter. A `Dispatch` is well-typed, arrives validated, and decodes cleanly. It is also, quite often, a lie. The reckoning prints the gap at the end:

```
15 dispatches were filed during the plan. Their type
guaranteed their shape. Nothing guaranteed their truth.
```

9 Summary of the guarantees

The compiler enforces	Run time must check
the reply type follows from the prompt	that the incoming tag matches its fields
which moves exist at which fibre (export by harvest grade, steel trade by position)	that a reply has the keys its fibre promises
that every fibre has a decoder	that field <i>types</i> are what they claim (only partly done)
that composites are well-formed	
Nothing enforces	
that a well-formed composite is ever <i>run</i> — <code>seqC(a,b)</code> is a value whether or not it is applied, and one such composite sat declared and unfired through a whole release;	
that the <code>delegate.ts</code> a hop spawns exists on disk;	
that anything a commissariat reports is true.	